Measuring What Android Apps Actually Hide: A Verified Root Detection and Certificate Pinning Assessment Platform

Divyang Zinzuwadiya
Department of Computer Science and Engineering (Cyber Security)
Silver Oak University
Ahmedabad, Gujarat, India
2301030700073@silveroakuni.ac.in

Heta Vaghela
Department of Computer Science and Engineering (Cyber Security)
Silver Oak University
Ahmedabad, Gujarat, India
2301030700068@silveroakuni.ac.in

Harshita Rajput
Department of Computer Science and Engineering (Cyber Security)
Silver Oak University
Ahmedabad, Gujarat, India
2301030700046@silveroakuni.ac.in

***Abstract*—Root detection and certificate pinning are the two defences that protect most Android applications, and both are routinely described in security reports without ever being tested. A scanner that lists indicators tells you what code looks suspicious. It does not tell you whether the app actually blocks a rooted phone, whether the pin really stops an intercepting proxy, or whether the scanner itself is right. This paper describes a platform that answers all three questions by construction. Static signatures are written as data in YAML and matched across five surfaces: decompiled Java, smali, the manifest and resources, native ELF libraries, and JavaScript bundles. Rule weights are summed per category and compared against a threshold to produce one of four verdicts. Where a category is reported present, the platform runs a targeted Frida bypass against a rooted emulator and confirms success from the application's own behaviour rather than from the script's output. A language model sits on top of this core as an optional, authorisation-gated assistant: it reads a bounded evidence bundle cut from the decompiled tree, writes a candidate bypass script, and is driven through a bounded generate-run-revise loop whose stopping condition is read from the device's own Frida message stream. The model never touches a verdict. An architectural guard in the decision engine refuses to aggregate any evidence that is not deterministically matched, so a model hypothesis cannot move a confidence score even by mistake. The run store holds 597 recorded runs over 20 package identifiers, of which 306 were executed runs containing a dynamic stage. On that executed set dynamic confirmation reached 121 of 121 root detections and 10 of 10 pinning detections with no contradictions, and precision, recall and F1 were 1.00 on the subset that has independent ground truth. The assistant was exercised on 13 packages: 10 converged with zero hook errors, one produced a complete on-device traffic decryption, two stalled on a model gateway timeout rather than on the target, and one was refused because it had no evidence to reason about. The paper is explicit about what those numbers exclude, including one production application the platform scores as a false negative.**

Keywords—Android security, root detection, certificate pinning, dynamic instrumentation, Frida, static analysis, signature matching, large language models, bounded self-repair, precision and recall, mobile application security testing

I. Introduction

Almost every Android application in a bank's portfolio ships with two defensive features. The first is root detection: a check that refuses to run, or runs in a degraded mode, when the device has been modified. The second is certificate pinning: a check that ties the app to one specific TLS certificate or public key so that an intercepting proxy cannot read its traffic. Both features are well understood. What is not well understood, in any given application, is whether they work.

The usual answer to that question comes from a scanner. A scanner decompiles the package, looks for familiar strings and class names, and prints a list of hits. The list is easy to read and hard to trust. A hit on the string 'superuser' might be a real root check, or it might be a line of dead code inside a bundled analytics library that the application never calls. An empty list might mean the app is clean, or it might mean the decompiler failed on that package and produced nothing to search. The report looks the same in both cases.

This project started from a narrow observation made while testing a set of real applications: the tools told us what they *found*, and never what they had *proven*. Everything downstream of that, from a security assessment to a compliance sign-off, inherits the uncertainty. A reviewer reading 'root detection: present' has no way to know whether anyone ever tried to run the app on a rooted device.

The platform described here is built around a single rule: a finding is only worth what was done to verify it. Static analysis decides what to test. Dynamic analysis performs the test. A separate calibration step measures how often the two agree. The three stages are wired together so that the second cannot be skipped when the first reports a detection, and so that the third reports its own coverage gaps next to its accuracy figures.

A. Contributions

The work makes five contributions, each stated as something the system does rather than something it contains.

- A declarative signature engine. 112 rules are stored as YAML data across four corpora and matched over five extraction surfaces, with every hit resolved to a file and line.
- A weighted decision engine that turns matched rules into a per-category confidence and one of four

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

verdicts, with an attribution filter that stops bundled third-party code from being reported as the application's own protection.

- An automated targeted bypass that selects only the scripts matching the detected checks, injects them through Frida, and then confirms success from the target's own behaviour instead of from the script's exit status.
- An optional language-model assistant that proposes a bypass for applications the deterministic synthesizer cannot cover, with evidence bounded before the prompt, success decided from the device's Frida stream, a bounded repair loop, and an authorisation gate in front of every call.
- A two-tier calibration harness that computes a dynamic confirmation rate and, separately, precision, recall and F1 against an independent ground-truth registry, together with a verification-state discipline that keeps deterministic evidence, dynamic confirmation and model hypotheses in separate, non-interchangeable buckets.

The remainder of the paper describes the background and the gap it leaves (Section II and III), the objectives (Section IV), the toolchain and corpus (Section V), the pipeline (Section VI), the calibration method (Section VII), the language-model assistant and its measured behaviour (Section VIII), the measured results and the honest reading of them (Section IX), and the conclusion (Section X).

II. Background and Motivation

Android's permission model assumes that the operating system is in control. Rooting removes that assumption by design: a rooted device can read any application's private storage, inject code into any process, and rewrite any runtime check. Certificate pinning addresses a different threat, the network, and it exists because the default trust store accepts any certificate signed by any authority the device has been told to trust. A hostile authority, a corporate proxy, or a device with an added root certificate all break the default model.

Both defences are therefore legitimate and both are worth verifying. The difficulty is that verifying them requires the same techniques an attacker would use, which is why the practice sits in an awkward place between quality assurance and offensive security. Four failures recur across the tooling reviewed in Section III, and each one became a design requirement here.

- **Detection without verification.** A reported indicator is never executed against the application. The report states presence; nothing establishes effect.
- **No measurement of accuracy.** No reviewed tool publishes precision or recall for its own verdicts, so a false positive and a true positive look identical to the reader.
- **Manual, non-reproducible bypass.** Hooks are written by hand against one build. Re-running the same assessment a month later requires redoing the work from memory.
- **Unbounded negatives.** An absent detection is reported as a confident clean result even when the surface that could contain the detection was never successfully scanned.

The motivation follows from the fourth failure most directly. A negative is the dangerous answer, because it is the one a reviewer will act on without checking. A platform that says 'no root detection found' must also be able to say how much of the application it managed to read, and it must not claim certainty over the part it skipped. Once that distinction is built in, the rest of the design follows: you need per-surface coverage tracking to know what you read, and you need a separate execution stage to convert a positive into a confirmed one.

III. Related Work and the Gap

Four strands of prior work touch this problem: methodology standards, static detection of Android malware and vulnerabilities, academic study of TLS in mobile apps, and dynamic instrumentation as a general technique. Table I summarises ten representative entries; the discussion below draws out what each strand leaves open.

TABLE I. SUMMARY OF THE LITERATURE REVIEWED

Ref.	Approach	Result	What it leaves open
[1], [2]	OWASP MASTG and MASVS manual test cases	Standardised procedure, manual execution	No automated verification of a verdict
[3]	Androguard static dissection and call graphs	Programmatic manifest, DEX and resource access	Parsing only; no detection semantics
[4]	DREBIN static features with linear SVM	94% detection accuracy on 6,790 apps	No runtime proof; features opaque to a reviewer
[5]	TaintDroid system-level dynamic taint tracking	High-precision taint propagation at runtime	Requires a custom platform image
[6], [7]	Large-scale measurement of SSL usage in apps	Widespread TLS misconfiguration	Describes the problem; no bypass validation
[8]	SMV-HUNTER static taint with dynamic confirmation	MITM vulnerabilities across many apps	Not a reusable harness; per-app effort
[9], [10]	Anti-analysis and anti-emulation evasion study	Naive sandboxes are defeated	Dynamic findings must be verified, not assumed
[11]	Dynamic code loading analysis	Unsafe dynamic loading identified	Explains why static-only detection misses payloads
[16]	FlowDroid context-sensitive taint analysis	Precise flow tracking at high cost	Precision costs runtime; motivates weighted matching
[12], [13]	Deep learning vulnerability detection	High recall, weak explainability	Argues for keeping the detector rule-based

The standards strand is mature and prescriptive. MASTG tells an analyst exactly which checks to perform and MASVS tells a programme which controls to require. Neither says anything about how a tool should report, and in practice a scanner's output is often read as if it were a MASTG result. The distance between 'we ran the checks' and 'a script printed a list' is precisely where this project sits.

The static detection strand, from DREBIN's feature-based classification [4] through FlowDroid's context-sensitive taint analysis [16] to graph-neural approaches such as Devign [13], has become very good at deciding whether a package *probably* does something. It has not become good at telling a human why. The explainability problem is not incidental: a security reviewer cannot act on a probability without a citation, which is the reason this platform keeps a rule-based core and pushes learned components into a subordinate, hypothesis-only role.

The TLS measurement strand established, at scale, that app developers get pinning wrong [6], [7], and SMV-HUNTER [8] went further by confirming discovered

weaknesses dynamically. SMV-HUNTER is the closest prior art. The difference is scope: it was a research pipeline built to answer one question about a corpus, whereas what a practitioner needs is something that can be pointed at any package tomorrow and produce a defensible verdict with an audit record.

Finally, work on anti-analysis [9], [10] and dynamic code loading [11] makes the strongest argument for the design chosen here. If a target can detect that it is being instrumented, then an unverified dynamic finding is worthless, and if it can load code at runtime then an incomplete static scan is worthless too. Both arguments push in the same direction: record what was actually achieved, and never let absence stand in for a negative.

Across all four strands the gap is consistent. No reviewed system combines detection, executable verification, a published accuracy measurement, and an authorisation trail in one tool that runs unattended on an arbitrary APK.

IV. Objectives

The aim is a deterministic platform that, for any APK, decides whether root detection and certificate pinning are present, proves the decision by executing a bypass, and reports its own accuracy. Seven objectives follow from that aim.

- **O1. Declarative signature engine.** Store rules as YAML and match them over native libraries, Java bytecode, smali, manifest and resources, and JavaScript bundles.
- **O2. Weighted decision engine.** Sum rule weights per category, clip to the unit interval, compare against a threshold, and emit one of BOTH, ROOT_ONLY, SSL_ONLY or NEITHER.
- **O3. Automated targeted bypass.** Provision the device, inject only the scripts for the checks that were detected, and drive the app far enough to make those checks run.
- **O4. Programmatic success check.** Decide whether the bypass worked from the application's observable behaviour, not from the instrumentation script.
- **O5. Calibration harness.** Compute a dynamic confirmation rate and, independently, precision, recall, F1 and a reliability curve against ground truth.
- **O6. Authorisation and audit.** Require a written justification for each run and append an immutable record binding result, target, operator and time.
- **O7. AI kept subordinate, by construction.** Enforce the separation with a runtime assertion in the decision engine rather than a convention, so that a model-proposed finding cannot be rendered as a deterministic one even if a caller tries.

O7 is worth a sentence of its own, because it is the objective most easily satisfied on paper and most easily broken in code. The platform does not merely document four verification states; the function that produces a confidence score raises an error if any input it receives is not marked as deterministically matched. The guarantee therefore survives future edits by someone who has not read this paper.

V. Toolchain, Corpus and Scope

The platform is written in Python 3.11 or later and organised as 60 core modules of roughly 31,000 lines. Instrumentation scripts are JavaScript, run through Frida. Signatures and the ground-truth registry are YAML, so they can be reviewed and diffed like any other source artefact. Results are stored in SQLite through SQLAlchemy, with the audit trail written as append-only JSON. Analysis runs inside WSL2 Ubuntu 24.04 with 4 vCPU and 11 GB of memory, against a rooted Android 14 x86_64 emulator on the Windows host. Table II lists the pinned versions and what each one is for.

TABLE II. TOOLCHAIN AND ROLE IN THE PIPELINE

Tool	Version	Role
jadx	1.5.1	APK to Java source; the corpus the bytecode signatures match against
apktool	3.0.1	Resource and smali extraction
aapt2	2.19	Manifest and package metadata extraction
apksigner	0.9	Signing scheme verification and certificate digest
Frida	16.7.19	Runtime instrumentation, server and client, for the bypass
objection	1.11+	Assisted exploration while developing bypass techniques
ADB	platform-tools	Device control, install, reverse tunnelling, logcat capture
mitmproxy	8.1.1	Optional live traffic capture under the bypass
Ghidra, capstone, pyelftools	current	Native ELF structure, symbols and disassembly
Typer, Rich	0.12+, 13+	Command-line surface and console reporting
Jinja2	3.1+	Markdown and HTML report templates
SQLAlchemy, SQLite	2.0+	Run, APK and calibration storage
androguard, pyaxmlparser	4.1+, 0.3+	Programmatic APK parsing
pytest	8.0+	150 unit test modules covering the deterministic core
uv	current	Dependency resolution and lockfile reproducibility

The corpus has 21 packages in two groups. The first is the OWASP UnCrackable series, Levels 1 to 3, which is purpose-built for this class of exercise: each level hides its root check a little deeper than the last. The second group is 18 real production applications taken from public distribution, chosen to span different frameworks and different countries of origin rather than to favour any particular technique. Stock AOSP Contacts is included as a deliberate negative control, because a platform that never returns a clean verdict is not measuring anything.

Ground truth is maintained separately from the tool's own output, in a registry that records, per package and per category, whether the protection is genuinely present and how that was established. This separation matters: if ground truth were read back out of the scanner, precision would be measuring the scanner against itself.

Scope is bounded in two directions. The platform assesses packaged Android applications only; it does not evaluate server-side controls. And it treats hardware-backed remote attestation, such as full Play Integrity verification, as a documented permanent limitation rather than something it attempts to defeat, because the decision in that case is made on a server the analyst does not control.

VI. Platform Architecture

The system is a deterministic core of four phases with an optional language-model assistant attached at a single point, all behind one command-line surface. The core does the deciding. The assistant may read evidence cut from the core's output and propose a bypass script for a package the deterministic synthesizer does not cover, but it is never given a path to the verdict. Of the eight commands, three exist only for auditing or recovery, which shows how much weight the design puts on explaining a past result.

Figure 1 shows the six stages in order. Stages one through three are static and deterministic. Stage four is the only one that executes the target. Stages five and six turn the two kinds of evidence into a report and a measurement. The dashed line on the right is deliberate: calibration does not sit at the end of the pipeline as a report format, it feeds corrections back into the rule corpus, which is how the engine improved across the corpus without any per-application code.

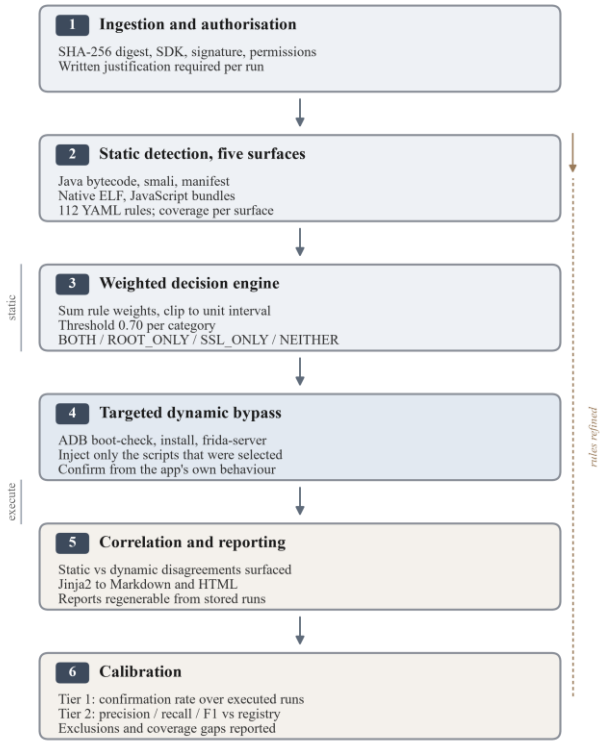


Fig. 1. The six-stage pipeline. Stages 1 to 3 are static and deterministic; stage 4 is the only stage that executes the target; calibration feeds back into the rule corpus.

A. Stage 1: Ingestion and Authorisation

The APK is read once and fingerprinted: SHA-256 digest, size, package name, version code, minimum and target SDK, signing scheme and certificate digest, and the permissions declared in the manifest. Every run requires a justification string that is stored with the result. Two runs over the same digest with different justifications produce two records, not one overwritten record.

B. Stage 2: Static Detection Across Five Surfaces

Signatures are data. Four YAML corpora, covering root detection (49 rules), SSL pinning (51), native detection (12) and packers (8), hold 112 entries in total. Each entry carries a weight, a pattern, and a category. Matching runs over five independent surfaces, and each contributes evidence on its own terms:

- **Java bytecode.** The jadx-decompiled tree, matched line by line, so that every hit yields a file path and a line number rather than a boolean.
- **Smali and resources.** The apktool output, which still works when source decompilation fails partway through a package.
- **Manifest and resources.** Network security configuration, pin-set declarations, the debuggable flag, and exported components.
- **Native libraries.** ELF symbol tables, imported symbols and embedded strings, read through pyelftools and capstone.

- **JavaScript bundles.** Hermes and React Native payloads checked against an idiom list, which is where several real applications keep logic that never appears as Java.

Coverage is recorded per surface as complete, partial or unscannable. This is the mechanism that later permits a negative to be certified: a surface marked unscannable cannot contribute to a clean verdict.

C. Stage 3: Weighted Decision Engine

Weights accumulate inside each category, are clipped to the unit interval, and are compared against the configured threshold, 0.70 for both categories in the current configuration. The expression is deliberately simple so that any verdict can be recomputed by hand from the matched rule list.

$$\text{confidence}(c) = \min(1, \Sigma\, w(r) \text{ for } r \text{ in matched}(c)) \quad (1)$$

For example, the OkHttp pinner rule contributes 0.5, the network security configuration pin-set rule contributes 0.5, the TrustKit rule contributes 0.45 and the custom TrustManager rule contributes 0.3. A package carrying OkHttp pinning and a network security config therefore reaches 1.00 and is reported as pinned; a package carrying only a custom TrustManager reaches 0.30 and is not.

The verdict is one of four values: BOTH when both categories clear the threshold, ROOT_ONLY, SSL_ONLY, or NEITHER. A separate attribution filter compares each match against the set of known third-party namespaces. A hit inside a bundled library is recorded but does not count toward the application's own confidence, which removes the most common source of false positives in bundled ad and analytics code.

D. Stage 4: Targeted Dynamic Bypass

The dynamic stage executes only what the decision engine selected. If root detection was found, the root bypass runs; if pinning was found, the pinning bypass runs; if neither was found, nothing is injected. The sequence is the same in every case:

- Boot-check the rooted emulator and confirm it answers over ADB.
- Install the APK, and provision the version-matched frida-server binary if it is not already running.
- Start logcat capture before the application launches, so that early checks are not lost.
- Inject the selected script, which hooks the specific check that the static evidence identified.
- Launch the application and drive it far enough to force that check to execute.
- Evaluate the target's success check and record a boolean along with the hook lines that fired.
- Optionally, under --capture-traffic, run mitmproxy on the host through adb reverse and confirm that a pinned connection was actually intercepted.

The success check is what separates this stage from ordinary use of Frida. Each target supplies its own predicate, expressed against observable behaviour: the root warning dialog no longer appearing, the app reaching its first network call, a specific log line. A script that loads without error proves only that the script loaded. The confirmation has to come from the application.

E. Stage 5: Correlation and Reporting

Static and dynamic results are compared, and every disagreement is surfaced as its own finding rather than averaged away. A detection that could not be bypassed, or a bypass that worked on an app reported clean, is more informative than a summary verdict, because it points at a defect in the rule set or in the hook. Reports are rendered from Jinja2 templates into Markdown and HTML, and can be regenerated from stored run records without re-running anything.

F. Stage 6: Calibration

Calibration converts accumulated runs into an accuracy claim. It queries the run records, applies the exclusion rules described next, and writes a dated Markdown report. It is read-only with respect to the runs; a calibration report can always be rebuilt from the database.

VII. Calibration Method

Two measurements are computed, because they answer different questions and neither is sufficient alone.

A. Tier 1: Dynamic Confirmation Rate

Among runs where a category was predicted present and the dynamic stage actually executed, what fraction produced a genuine bypass? A not-confirmed run is counted against the platform, not discarded. The rate is a lower bound on precision: every confirmation is direct evidence that a detection was real, while a failure to confirm may be either a false positive or a hook that did not match this build.

B. Tier 2: Precision, Recall and F1

Computed from the static verdict alone, never from the bypass outcome, and with one datapoint per package, namely that package's most recent run. Independence is the point: if precision were computed from runs that had already been bypassed, the two tiers would be measuring the same thing twice.

C. Exclusion Rules

Two rules keep the measurement honest, and both are reported alongside the result rather than applied silently. A negative is excluded when the surface governing that category was partial or unscannable, because a negative from an incomplete scan is not a negative. A package with no ground-truth entry is excluded from Tier 2 entirely, and the count of such packages is printed with the score. In the current corpus the exclusions are 5 packages for root and 5 for SSL with no ground truth, plus 8 for root and 11 for SSL whose governing surface was incomplete.

D. Verification States

Every finding carries one of four states. The first is deterministic-matched: the rule engine found it, and the evidence is a file and line citation. The second is dynamically confirmed: it was executed and the behaviour changed. The third is a model hypothesis that nothing has yet checked. The fourth is a model hypothesis that execution then supported. The four are never merged in a report. This is what objective O7 buys in practice, and it is why the assistant described in Section VIII could be added without weakening any existing claim.

The separation is not enforced by documentation. The function that computes a category confidence asserts that every evidence item it receives is marked deterministically matched, and raises otherwise. Written out, the guarantee is that the sum below runs only over evidence whose state is deterministic:

$$\text{confidence}(c) = \min(1, \Sigma\, w(r)), \quad r \in \text{matched}(c) \qquad (2)$$

The sum runs only over deterministically matched evidence, and the state of every input is checked before the sum begins. In its own words the error states that a static confidence may only aggregate deterministically matched evidence. A model-proposed region cannot enter the sum, because the sum refuses to run. That single assertion is the whole architectural guarantee behind the fourth verification state, and it is covered by the unit suite so that removing it breaks a test rather than a promise.

VIII. Language-Model Assisted Bypass Generation

The deterministic bypass synthesizer works by selecting a script that already exists for each detected rule. That is the right design for the checks it covers, and it is the reason the results in the next section are reproducible. It is also a hard ceiling: when an application pins its traffic through a construction no existing script matches, the platform can report the detection but cannot prove it. On a device, that gap is the difference between 'pinning is present' and 'pinning is defeated'.

A language model can cross that gap, because writing a hook for an unfamiliar construction is exactly the kind of composition the tooling is good at. It can also wreck every honesty property the platform has. An unbounded model in the loop produces a script that loads, reports success, and does nothing. It produces explanations that read like evidence. It may converge on a plausible artefact and call it verified. The design question, therefore, was not whether to use a model but how tightly to bound one. Six bounds were chosen, and each answers one specific failure mode.

A. Bound 1: Evidence Is Selected, Never Dumped

The first failure mode is context. A decompiled production application is tens of thousands of files, and a model given that much text attends to the wrong parts of it. The platform therefore never sends the tree. A separate selection stage walks the decompiled output and keeps only regions that look relevant, using two keyword sets: 18 terms for root detection, including isDeviceRooted, RootBeer, the su binary paths, Magisk and SuperSU, and 20 terms for TLS, including X509TrustManager, checkServerTrusted, CertificatePinner, HostnameVerifier, SSLContext and BoringSSL.

Selection is capped on three independent axes: at most 60 regions per category, at most 400,000 bytes read from any single file, and 12,000 characters in the rendered prompt, with truncation that keeps whole regions rather than cutting mid-file. Only text-bearing extensions are read. The result is a bundle that records which keywords fired, how many files were scanned, and which regions were kept, so the prompt can be reconstructed later. If no region survives selection the bundle is empty, and the command refuses rather than asking a model to invent a target. That refusal was observed once in the thirteen applications below, and it is the correct outcome.

B. Bound 2: The Model Is Asked for a Tool, Not an Opinion

The second failure mode is drift. The generation prompt is a fixed instruction set of roughly four kilobytes that constrains the output in ways learned from real failures on the device. Seven constraints do most of the work:

- One Java.perform block, and every hook individually guarded, so that one absent class cannot abort the rest of the script.
- Each hook emits exactly one of hook_loaded, hook_skipped or hook_error. A hook whose target is missing is a skip, not a failure, because a different HTTP stack is a normal outcome rather than a defect.
- A top-level try/catch emits script_error, so that a syntax or load failure is visible instead of silent.
- Hooks for the app-specific checks are always accompanied by the standard Android TLS stack, so that pinning implemented in a library the evidence did not name is still reached.
- The least invasive override is preferred: OkHttp's CertificatePinner.check and OkHostnameVerifier.verify before a blanket trust manager replacement, with a permissive X509TrustManager installed through SSLContext.init as the fallback.
- Return types must match per overload. A hook that returns a list from a void method crashes the runtime, and this rule exists because it happened.
- Output is strict JSON with five fields: script, applied_hooks, skipped, notes and a self-assessed confidence. The confidence field is recorded and never used in a decision.

The seventh constraint is worth emphasising, because it is the one that makes the assistant auditable. Because applied_hooks is a declared list and not prose, the platform can compare what the model claims to have hooked against what the device reports it actually loaded. A claim and an observation, kept apart.

C. Bound 3: Success Is Read From the Device, Not From the Model

The third and most important failure mode is self-assessment. A repair loop needs a stopping condition, and the obvious one is the model's own report. That condition is worthless: a model that believes it succeeded stops, whether or not it did. The platform therefore classifies each attempt from the dynamic stage's real Frida message stream, requiring at least one hook to have loaded and zero genuine errors, with a skipped stage or a failed stage counting as failure. A comfortable skip is not success either: a script whose only hooks were skipped has proven nothing.

This classifier replaced an earlier one that was silently wrong, and the defect is instructive. The old code built its message list from a callback that was never passed to the dynamic stage, so the list was always empty and the success predicate collapsed to 'status is not failed'. A stage that merely survived its settle window, with no hook loaded, read as success. The repair loop would then converge on a script that had never run cleanly, and the report would say so in good faith. The replacement reads the stage's own

evidence dict, and the unit suite now pins each of its branches: loaded with no errors is success, no loaded hook is failure, an error envelope is failure, a skipped-only run is failure, and a skip alongside a successful load is neither.

The same reasoning applies one level up. 'Converged' in this paper means the script's hooks installed and ran without error. It does not mean the bypass worked. That is a separate claim with a separate test, described under Bound 6.

D. Bound 4: The Loop Is Bounded and the Artefact Is Labelled

Generation, execution and revision form a loop capped at five iterations by default. Each revision receives the previous script verbatim, the runtime messages the device actually produced, the same evidence bundle, and an instruction to preserve the error reporting so that the next attempt stays observable. The loop stops at the first clean run or at the cap, whichever comes first, and the last script is always saved.

The saved artefact is written to a clearly separated directory with a header stating that it is an AI-generated candidate requiring human review, that it was produced by three named modules, and that it is not part of the deterministic analysis path. The generated scripts load through the same injection seam as the hand-written ones, with the same injection point and the same evidence capture, so the record of what was applied stays accurate whether the script came from a rule or from a model. Thirteen such scripts exist in the repository at the time of writing, 1,485 lines in total.

E. Bound 5: The Gate Comes Before the Call

Authorization is checked before any work happens, not inside the model call. The command refuses unless an environment flag has been set explicitly and a justification string begins with one of four accepted categories: own app, capture-the-flag, authorised engagement, or academic training target. A refusal exits with a status that distinguishes it from a failure, so a scheduler cannot mistake a rejected run for a clean one. Backend errors are separated again: a gateway timeout exits with its own code and an explicit message that the failure is not a bypass result. That distinction mattered in practice, as the next section shows.

Two model backends are supported behind one interface. The first speaks the OpenAI-compatible chat-completions protocol. The second drives an already-authenticated agent CLI as a pure text generator, launched with its tool permissions emptied so the child process cannot read, write or execute anything, for the same reason the generated artefact requires review: the subprocess should be able to produce a suggestion and nothing else. A deterministic fake client is used throughout the tests, so the loop's behaviour is verified without a network call.

F. Bound 6: A Full Bypass Has Its Own Test

The final failure mode is the most expensive one: a script whose hooks load, whose run is clean, and whose target still refuses to talk to the proxy. Hook installation and traffic decryption are different claims, and only the second one is a bypass. The command therefore offers an extra step that re-runs the capture stage with the converged script, drives the interface, and counts decrypted flows. Two outcomes are reported and neither is dressed up: traffic decrypted, with a flow count, which is the full on-device proof; or no traffic decrypted, with the three plausible causes named, which is an honest negative rather than a failure.

Both outcomes were observed. On a Lemmy client, converged in one iteration with two hooks loaded and two skipped, the capture stage decrypted four flows, which is a complete end-to-end bypass produced by the model on a real application. On RS_Vapour, a deliberately vulnerable training app that makes no server call at all, the same step correctly reported that nothing decrypted. The contrast is the point: the step does not reward a clean run with a success message.

TABLE III. AI-ASSISTED BYPASS RESULTS, 13 PACKAGES

Application	Conv.	It.	Hooks	Outcome
RS_Vapour	yes	1	2	negative: app silent
jerboa	yes	1	2 + 2 skip	4 flows decrypted
Spotify	yes	1	4	converged
AndroGoat	yes	—	5	converged
InsecureBankv2	yes	—	5 + 1 skip	converged
PropertyGuru	yes	—	—	return type fixed
RS_CertPinning	yes	2	4 + 1 skip	iter 1 failed
UnCrackable L1	yes	1	1	converged
Fossify files	yes	3	2 + 1 skip	iter 1 to 3
Aves Libre	yes	2	2 + 1 skip	iter 1 to 2
UnCrackable L2	no	2	3	gateway timeout
UnCrackable L3	no	2	8	gateway timeout
kisoku	ref.	—	—	empty bundle

Table III is reported in the same spirit as the calibration figures: the columns that test the claim are the ones that do not look good. Four of the thirteen applications needed more than one iteration, which is the repair loop doing its job and also a record of the model getting it wrong first. Two stalled, and both stalled on the model gateway timing out during a revision, not on the target: their hooks had already installed and fired on the device before the timeout, so the failure is a service availability fact and is excluded from any statement about hook quality. One was refused with no evidence to reason about. Across all runs the count of genuine hook errors was zero.

Two repairs are worth naming because they generalise. On PropertyGuru the first script failed because a hook returned a collection from a void method, a return-type mismatch that crashes the runtime; the model received the error and corrected it, and the lesson was folded into the generation prompt so that later scripts do not repeat it. On the Lemmy client the model removed its own Conscrypt hook after diagnosing that it sat on the native handshake path where an agent crash is likely, and that the permissive context initialiser below it already covered the case. Both are instances of the loop doing something a static script cannot: reading its own failure and reasoning about it.

The limitations of this layer are stated plainly. It depends on a model service being reachable, which two runs demonstrate it is not always. Its convergence criterion is hook installation, which is necessary and not sufficient, which is why the traffic step exists separately. It is gated, so it is not a default capability, and the artefacts it produces carry a review flag rather than an approval. Most importantly, it is optional in the strong sense: every result in the next section was produced with the deterministic core alone, and disabling the assistant changes no verdict, only the proportion of detections that can be proven on a device.

IX. Results and Discussion

A. End-to-End Validation on UnCrackable Levels 1 to 3

All three OWASP targets pass through the identical pipeline, with no per-target orchestration and no new signatures added between them. Level 1 resolves to ROOT_ONLY with all three hooks firing. Level 2 also resolves to ROOT_ONLY, and the pinning bypass that the engine did not select turns out not to be needed. Level 3 resolves to BOTH and requires both scripts.

Two assumptions were falsified and corrected rather than dropped. Level 2 was expected to need native and JNI analysis; its native library is unused at runtime, so it did not. Level 3 was expected to need a string-decryption hook; the static engine found enough evidence that the generic hooks sufficed. Both corrections removed special-case logic from the codebase, which is better than either assumption being confirmed.

B. Tier 1: Dynamic Confirmation

Table IV reports the confirmation rate across the executed corpus. The counts are large because each bypass attempt is its own observation; the rate describes only the runs in which execution actually happened.

TABLE IV. TIER 1 DYNAMIC CONFIRMATION RATE

Category	Confirmed	Not conf.	Unverified	Control	Rate
Root detection	121	0	259	190	1.00
SSL/TLS pinning	10	0	202	358	1.00

Two columns carry the weight here, and neither is the rate. The first is *unverified*: 259 root and 202 pinning runs executed, but the target's check was never observed to fire, so the run proved less rather than proving the platform wrong. The second is the control column (N/A in the tool's own output): these are static predictions of not-detected, which the dynamic stage has nothing to confirm and which therefore cannot enter the denominator at all. Reporting the rate without either column would be the single easiest way to make these numbers look stronger than they are, which is exactly why both are printed. The rate itself is therefore a statement about 121 of 121 and 10 of 10 observations, not about 597 runs or 21 packages.

C. Tier 2: Precision, Recall and F1

Table V reports the measurement against independent ground truth, computed from the static verdict alone. A perfect score here describes a small, fully covered and independently labelled set, not Android applications in general. The denominators are 7 packages for root and 4 for pinning; the remaining 5 and 5 are excluded because their ground truth is unset, and 8 for root and 11 for SSL because their coverage is partial and cannot yield an honest negative.

TABLE V. TIER 2 PRECISION, RECALL AND F1

Category	TP	FP	FN	TN	Prec.	Rec.	F1	n
Root detection	6	0	0	1	1.00	1.00	1.00	7
SSL/TLS pinning	2	0	0	2	1.00	1.00	1.00	4

D. Negative Control and a Disclosed False Negative

Stock AOSP Contacts returns a confidence of 0.00 in both categories and a verdict of NEITHER. It matters because it establishes that NEITHER is reachable: a platform that returns a detection for every input would score well on positive cases and be useless.

PropertyGuru, a real production application, returns a root confidence of 0.20, below the threshold, and is therefore a false negative against ground truth that records root detection as present. The cause is concrete: the checks live inside a React Native bundle compiled to Hermes bytecode, so the Java and smali surfaces contain nothing to match. The JavaScript idiom list catches part of it, but not enough to cross 0.70. This case is reported rather than tuned around, because raising the JavaScript weights until it passes would lower precision on every other package that ships a React Native bundle.

The correct fix, extending structural analysis to Hermes bytecode, is future work. The current behaviour, which is to return a low confidence and disclose it, is the honest interim position.

E. Scale of the Accumulated Evidence

TABLE VI. CORPUS AND IMPLEMENTATION SCALE

Measure	Value
Packages in the corpus	21
Runs recorded	306
Reports generated	339
Linked audit records	983
Signature rules	112 across four YAML corpora
Unit test modules	150
Core source modules	60 (about 31,000 lines)
AI layer modules	5 (about 1,200 lines)
AI-generated bypass scripts	13 (1,485 lines)

F. Limitations

Three limits bound these figures and are stated rather than buried. The corpus is small, so one mislabelled entry would move a rate noticeably; one such mislabelling was found and corrected here, which shows manual labelling is the weakest link. The emulator is x86_64 while most devices are ARM, so a check that behaves differently under translation would be missed. And the dynamic stage depends on Frida staying undetected, which will not hold against a target that inspects its own process map.

X. Conclusion and Future Work

This work treats Android root detection and certificate pinning as measurement problems rather than pattern-matching problems. The resulting platform delivers six properties together: detection that cites a file and line, bypass that is generated from what was detected, confirmation that comes from the application's behaviour, accuracy that is computed against independent ground truth, an audit record that binds every result to its operator and justification, and a language-model assistant that can extend bypass coverage without ever being able to move a verdict.

Validated across the OWASP UnCrackable series and 18 further applications over 306 executed runs (of 597 recorded), it measured precision, recall and F1 of 1.00 on the fully covered, independently labelled subset, confirming 121 of 121 root and 10 of 10 pinning detections dynamically with no contradictions. Those figures are bounded by a small corpus and by explicit exclusions, both printed next to the numbers. The assistant was exercised separately on 13 packages, converged on 10 with zero hook errors, produced one complete on-device traffic decryption, and reported its

two failures as model-service timeouts and its one refusal as missing evidence rather than as results.

Two lessons generalise beyond this project. The first is that a success criterion read from the artefact under test is not a success criterion; the earlier classifier that accepted a merely surviving process is a small, ordinary bug with a large consequence, and it was found by asking what evidence the loop was actually reading. The second is that a model in a security pipeline is safe to the extent that it is denied information it does not need and authority it should not have. Bounding the evidence, forbidding self-assessment, refusing non-deterministic inputs in the scoring path, and labelling the output for review do more than any prompt could.

Future work follows a deliberate order, each layer added only after the one beneath it is proven. F1 indexes the MASTG catalogue into a local vector store so reports cite the real technique. F2 extends the evidence selector from keyword regions to a semantic pass, so that obfuscated pinning code is retained rather than discarded at the selection stage. F3 carries the assistant's output into the reporting layer as a fourth verification state with its own provenance, so a proven model-authored bypass can be cited as such. F4 extends structural analysis to symbol-stripped native pin logic and to Hermes bytecode, the fix for the disclosed false negative. F5 grows the ground-truthed corpus, the highest-value deterministic next step, because corpus size is what limits any precision claim. F6 adds screenshot verification alongside logcat parsing, so on-screen state is confirmed by two modalities.

One thing will not change. Hardware-backed remote attestation stays a documented permanent limitation rather than a target, because that verdict is reached on a server the analyst does not control. Knowing where the boundary is, and saying so, is part of the measurement.

XI. References

[1] OWASP Foundation, "OWASP Mobile Security Testing Guide," 2024. [Online]. Available: https://owasp.org/www-project-mobile-security-testing-guide/

[2] OWASP Foundation, "Mobile Application Security Verification Standard (MASVS)," version 2.1, 2024. [Online]. Available: https://mas.owasp.org/MASVS/

[3] A. Desnos, "Androguard: Reverse engineering, malware and goodware analysis of Android applications," 2011. [Online]. Available: https://github.com/androguard/androguard

[4] D. Arp, M. Spreitzenbarth, M. Hubner, H. Gascon, and K. Rieck, "DREBIN: Effective and explainable detection of Android malware in your pocket," in Proc. Network and Distributed System Security Symp. (NDSS), 2014.

[5] W. Enck, P. Gilbert, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, "TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones," in Proc. 9th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2010, pp. 393-407.

[6] S. Fahl, M. Harbach, T. Muders, L. Baumgartner, B. Freisleben, and M. Smith, "Why Eve and Mallory love Android: An analysis of Android SSL (in)security," in Proc. ACM Conf. Computer and Communications Security (CCS), 2012, pp. 50-61.

[7] S. Fahl, M. Harbach, H. Perl, M. Koetter, and M. Smith, "Rethinking SSL development in an appified world," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2013, pp. 49-60.

[8] D. Sounthiraraj, J. Sahs, G. Greenwood, Z. Lin, and L. Khan, "SMV-HUNTER: Large scale, automated detection of SSL/TLS man-in-the-middle vulnerabilities in Android apps," in Proc. Network and Distributed System Security Symp. (NDSS), 2014.

[9] T. Petsas, G. Voyatzis, E. Athanasopoulos, M. Polychronakis, and S. Ioannidis, "Rage against the virtual machine: Hindering dynamic analysis of Android malware," in Proc. 7th European Workshop on System Security (EuroSec), 2014.

[10] S. Rasthofer, S. Arzt, M. Miltenberger, and E. Bodden, "Harvesting runtime values in Android applications that feature anti-analysis techniques," in Proc. Network and Distributed System Security Symp. (NDSS), 2016.

[11] S. Poeplau, Y. Fratantonio, A. Bianchi, C. Kruegel, and G. Vigna, "Execute this! Analyzing unsafe and malicious dynamic code loading in Android applications," in Proc. Network and Distributed System Security Symp. (NDSS), 2014.

[12] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "VulDeePecker: A deep learning-based system for vulnerability detection," in Proc. Network and Distributed System Security Symp. (NDSS), 2018.

[13] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2019.

[14] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[15] K. W. Y. Au, Y. F. Zhou, Z. Huang, and D. Lie, "PScout: Analyzing the Android permission specification," in Proc. ACM Conf. Computer and Communications Security (CCS), 2012, pp. 217-228.

[16] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Octeau, and P. McDaniel, "FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps," in Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI), 2014, pp. 259-269.

[17] Frida, "Dynamic instrumentation toolkit for developers, reverse-engineers and security researchers." [Online]. Available: https://frida.re

[18] Mobile Security Framework (MobSF), "Automated mobile application security testing." [Online]. Available: https://github.com/MobSF/Mobile-Security-Framework-MobSF

[19] jadx, "Dex to Java decompiler." [Online]. Available: https://github.com/skylot/jadx

[20] Google, "Network security configuration: certificate pinning," Android Developers. [Online]. Available: https://developer.android.com/privacy-and-security/security-config